PySpark Playground

Plataforma web interactiva para el aprendizaje práctico de Apache PySpark en el navegador.

Versión: 1.0.0 | **Actualización:** Septiembre 2026 | **Ejercicios Totales:** 35

1. Stack Tecnológico Principal

Tecnología	Versión	Función en la Plataforma
HTML5 / CSS3	-	Estructura semántica, diseño responsive y estilos globales.
JavaScript Vanilla	ES6+	Lógica de la aplicación, estado y gestión de eventos.
Bootstrap	v5.3.3	Layout de navegación mobile, pestañas de editor y retícula.
Pyodide	v0.26.4	Entorno de Python en WebAssembly para la validación de tests.
Monaco Editor	v0.45.0	Editor de código interactivo con resaltado de sintaxis (VS Code).
Apache PySpark	3.5.0	Framework Big Data simulado en entorno cliente.
Service Worker	-	Caché de activos estáticos con estrategia Network-First.

2. Niveles de Aprendizaje y Contenido

Principiante

12 Lecciones

SparkSession, DataFrames, show(), select, filter, withColumn, dropna, orderBy, dropDuplicates.

Intermedio

10 Lecciones

Limpieza de datos, groupBy, joins, SQL, exportación de formatos CSV/JSON/Parquet.

Avanzado

10 Lecciones

Python UDFs, UDTFs, Spark UI, monitoreo (top/ps), profiling, stacktraces y debugging.

Desafío Final

3 Retos

ETL Completo, Análisis de Ventas y Pipeline de Datos integrados.

3. Estructura del Proyecto

```
Pyspark Playground/ ├── index.html # Landing principal ├── principiante.html # Grid principiante ├── intermedio.html # Grid intermedio ├── avanzado.html # Grid avanzado ├── desafio.html # Desafío final (3 retos) ├── lecciones_*.html # Playgrounds (32 lecciones) ├── sw.js # Service Worker (Network-first) ├── vercel.json # Configuración de hosting └── README.md # Documentación técnica
```

Arquitectura de Archivos Estáticos:

El proyecto adopta una arquitectura 100% Jamstack sin servidor backend. La navegación entre módulos se realiza mediante páginas de grid especializadas que leen el estado de avance desde el navegador.

Las lecciones compartidas utilizan vistas estandarizadas para mantener consistencia visual y de rendimiento.

4. Funcionalidades Principales de la Plataforma

Editor de Código & Terminal

- **Monaco Editor:** Resaltado de sintaxis Python, números de línea, autocompletado y atajos (`Ctrl+Enter` para ejecutar).
- **Panel Terminal:** Salida formateada con tablas ASCII PySpark (`+---+`), soporte para `.printSchema()` y `.describe()` .

Evaluación & Progreso

- **Sistema de Tests:** Validación en tiempo real mediante string matching ejecutado en Pyodide (WebAssembly).
- **Persistencia Local:** Control de progreso guardado en `localStorage` con barra de estado por nivel.

5. Arquitectura del Simulador de PySpark

Dado que Apache PySpark requiere un entorno JVM y Py4J no ejecutable nativamente en WebAssembly, la plataforma incorpora un **Engine de Simulación en JavaScript** que parsea las sentencias del usuario generando salidas fidedignas:

Operación Evaluada	Mecanismo y Salida Simulada
`df.show()`	Genera una representación tabular ASCII estructurada con bordes `+---+` .
`df.printSchema()`	Imprime el árbol de esquema formateado bajo la raíz `root` .
`.groupBy() / .join()`	Procesa relaciones simples de datos mock y devuelve vistas resumidas.
`spark.read.* / SQL`	Simula la lectura de datasets conceptuales y ejecución de queries SQL simples.

6. Estrategia SEO, PWA y Despliegue

Caché y PWA (Service Worker)

Se emplean 4 almacenes de caché independientes para garantizar un rendimiento óptimo:

- `pyspark-playground-v1` (App Shell)
- `pyodide-v0.26.4` (Binarios WASM)
- `monaco-v0.45.0` (Librerías del editor)
- `python-packages-v1` (Módulos auxiliares)

Indexación y Estructura SEO

- **Metadatos:** Integración de Open Graph y Twitter Cards en todas las landing pages.
- **Indexación Controlada:** `robots.txt` permite la indexación de las páginas de grid y bloquea las lecciones individuales (`noindex`).
- **Schema.org:** JSON-LD configurado bajo la entidad `WebApplication` .

7. Instrucciones de Despliegue Local

Al depender de Service Workers y Pyodide WASM, la aplicación debe servirse obligatoriamente a través de un servidor HTTP local (evitando el protocolo `file://`):

`python -m http.server 8000` o `npx serve .`